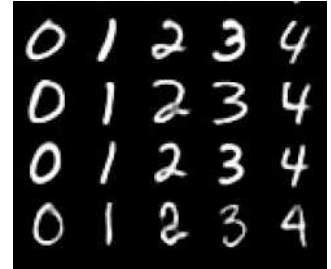


Introduction to Deep Learning - Exercise #2

submission date: 20/5/2025

Programming Task: Classifying and encoding the MNISTdigit dataset.

The MNIST dataset consists of 60,000 (+10,000 test) small images of scanned hand-written digits (0-9). The dataset contains the digits values as labels. The original images are 28-by-28 pixels. The images are monochromatic, i.e., have a single brightness channel with values between zero (black) and one (white).



In this exercise we will design and train both autoencoding and classification CNN networks.

Specific tasks:

For every section that includes training, paste the code of the training loop at the end of the section. If there are inner functions, you don't need to paste them, as long they have clear, informative names.

1. **Autoencoder.** Define a convolutional autoencoder to encode (and decode) the images through a small dimensional latent space.
 - a. How will you make the architecture flexible enough to choose any latent dim d , regardless of the number of channels chosen?
 - b. Explore two network configurations, one with a small number of channels (around 4 in the first layer, and similar numbers in the following layers) and a larger one (around 16 in the first layer) and report these tests as well; input vs. reconstructed images and the reconstruction loss obtained.
 - c. Explore the loss you get for $d=4/16$, and rationalize it in your report.
 - d. Describe and explain the network architecture you choose for this particular data (stride factors / #layers / #filters in each layer / non-linearity used).

The best practice of implementing this code is by defining separate encoder and decoder models, both inheriting from `nn.Module`. Use a mean L1 error to define the reconstruction loss.

2. **Classifier.** Use the same architecture as the encoder in Q1 in order to define a classification network by combining it with a single-layered MLP network that will map the latent space into a 10 (digits) class prediction. Train this network to predict the digit classes using cross-entropy loss in two scenarios:
 - (i) over the entire training set,
 - (ii) over only 100 random training examples.

Plot the training and test errors as well as accuracies. The best practice here is to keep the model divided into two models: the original encoder model and the added MLP model.

3. **Pre-trained Representation.** Repeat the two tests in Q2, this time by using the unsupervised pre-trained encoder weights from Q1 as a fixed (non-trainable encoder model), and only train the final MLP, once with the entire dataset, once with only 100 random training examples. Plot the training and test errors as well as accuracies. How do they compare to the ones in Q2? Write your conclusion as to the usefulness of unsupervised representation learning.
4. **Task Specific Encoding:** both Q1 and Q2 produce two different trained encoding networks. The one in Q1 already has a matching trained decoder, but the one in Q2 does not. Use this pretrained encoding network as a fixed (non-trainable model) and train a matching decoder over the entire dataset. Meaning that you are using an encoder trained for classification task, and coupling it with a decoder trained for reconstruction (trained from scratch along with the pretrained encoding/classifying network from Q2). Look at reconstructed images produced by these two sets of encoder-decoder networks and answer:
 - a. Which one results in better reconstruction error and why?
 - b. Describe the qualitative differences between the reconstructed results they produce. Explain why you think these are the differences.
 - c. Where do you see higher in-class (per-digit) variability? (watch and show multiple instances of the same digit to answer this)
 - d. Where do you see higher inter-class (between digits) distance/separation? Explain both.

To save you some time, here's how you define a subset of the training set:

```
train_loader = datasets.MNIST("data",
                             train= True,
                             download=True,
                             transform = transform)

indices = torch.arange(100)
train_loader_CLS = data_utils.Subset(train_loader, indices)

train_loader_CLS = torch.utils.data.DataLoader(train_loader_CLS,
batch_size=batch_size,shuffle=True, num_workers=0)
```

Moreover, some of the tasks above require you to “freeze” models from training. This can be achieved in different ways:

- going through all the weights in the network and setting `w.trainable=False`,
- simply excluding these models from the optimizer
- applying the “frozen” models under `with torch.no_grad()`:

We are expecting you to report and elaborate on every practical task in the pdf, with your own words and analysis of what you've done. Include everything that you think is crucial for us to understand your way of thinking.

Theoretical Questions:

- 1) **LTI**. Show that a convolution with respect to any filter h is time/space invariant.
- 2) **TI**. Explain whether each of the following layers are time/space invariant or not:
 - a) Additive constant
 - b) Pointwise nonlinearity (such as ReLU)
 - c) Strided pooling by a factor > 1
 - d) As a result, is a CNN composed of all these operators (+convolution) time invariant?
- 3) **Layers' Jacobian**. Calculate the Jacobian matrix of the following layers:
 - a) Additive bias vector
 - b) General Matrix multiplication
 - c) Convolution layer

Submission Guidelines:

The submission is in **pairs**.

Please submit a single pdf named `ex2_ID1_ID2_answers.pdf` in the dedicated submission box. This file should contain your answers to the theoretical part as well as the figures/text for the practical part.

Separately, please submit your code files compressed as zip file, named "`ex2_ID1_ID2.zip`", the dedicated submission box. Furthermore, include in this compress file a README with your names and cse usernames

Note: you must register as a pair in **both** the submission boxes. Please avoid inconsistencies.

Please write readable code, with documentation where needed, as the code will also be checked manually.

Late submission - 10 points reduction for each day. Submissions will not be accepted after 4 days.

Alone submissions - only to the very few students that got explicit approval.